Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Cloud Computing

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Serverless computing as a phrase – what does it mean?
- What is The Cloud?
- Some examples of real world IaaS, PaaS and FaaS providers
- Cloud architectures
- Containers
- Edge Computing and The Fog
- Benefits and Drawbacks of Cloud computing

UNIVERSITY of HULL

Serverless Computing

- The cloud is often described as ‘serverless computing’

- Serverless Computing is a misnomer as there are huge numbers of servers, but…
  - The company using The Cloud doesn’t *need to* own, manage or maintain the server(s)
  - Nor are they likely to use all of the resources on a server (in many cases)
  - They don’t care which physical server they use and if a server fails it can immediately fail over to a new physical machine

- There are many ‘Clouds’
  - Typically a Cloud is owned and operated by a single company
  - However, they may interoperate
  - Or be owned by a consortium

- Clouds can be private or public
  - Or a combination of the two

UNIVERSITY of HULL

What is The Cloud?

- The Cloud is simply a network of one or more servers which offer different remote functions
- These servers allow the services we looked at in the last session (e.g. SaaS, PaaS, IaaS) to function, often at a large scale, for many concurrent applications/users.
- Typically, servers have a specific purpose depending on their role, e.g.
  - Data storage (either filesystem or database)
  - Routing, and request management (balancing and routing message flow)
  - Specific processing (processing for specific purposes, e.g. a single application or specific functions)
  - General processing (dynamic reassignment to process different application functions)
- Aims
  - Scalability
  - Rapid deployment
  - Reduced maintenance (and cost?)

UNIVERSITY of HULL

Virtual Machines

Virtual Machines give access to what seems to be a full computer

- Actually is shared resources on a single machine acting as multiple machines
- Each Virtual Machine is isolated

- Control over entire virtual system
  - Install custom software and frameworks
  - Change OS-level settings
  - Run any software or service (including malware…)
  - Same management and security issues as local servers systems (although some firewalling may exist)

- Conventional VMs may take some setup time
- Must sit on top of a Virtual Machine management solution (e.g. hypervisor) which shares resources
- If the physical machine or hypervisor breaks all VMs break

UNIVERSITY of HULL

Cloud IaaS – Real World Examples

Many differing IaaS Data and Compute offerings for different purposes, including:
Amazon Elastic Compute Cloud (EC2) | Google Compute Engine (GCE)

- Realistically, smart virtual machines for automatic creation, failover, scalability and location choice

  - (Almost) instant virtual servers / Various real-world locations available
    - Create new ones very quickly to satisfy demand
    - Can be automatic with load balancing
  - Billing by the resources available / according to capacity
  - Control over some aspects of virtual environment (often OS/frameworks)
    - Run any software, service or store any data (including malware…)
    - Generic management and security issues often handled but lots of options available to developer
  - Can make snapshots and create copies for scalability

  - Used by (for example) Netflix
    to manage usage fluctuation

UNIVERSITY of HULL

PaaS – Real World Examples

- Amazon Elastic Beanstalk
  - Supports applications developed in Java, PHP, .NET, Node.js, Python and Ruby
- Google App Engine
  - Supports applications developed in Python, Java, Go, Node.js, PHP, .NET and Ruby
- Azure Web Services
  - Supports applications developed in Python, Java, Node.js, PHP, .NET, .NET Core and Ruby

- Developer doesn't manage the infrastructure that runs the applications
  - Lower overhead but less control (e.g. OS / Security / Resources)
- When an application is deployed, 'smart' PaaS offerings automatically host it using the required infrastructure and framework
- Application is usually persistent (unlike FaaS)
  - But may be dormant
- Billed in compute hours (actual resource usage)

UNIVERSITY of HULL

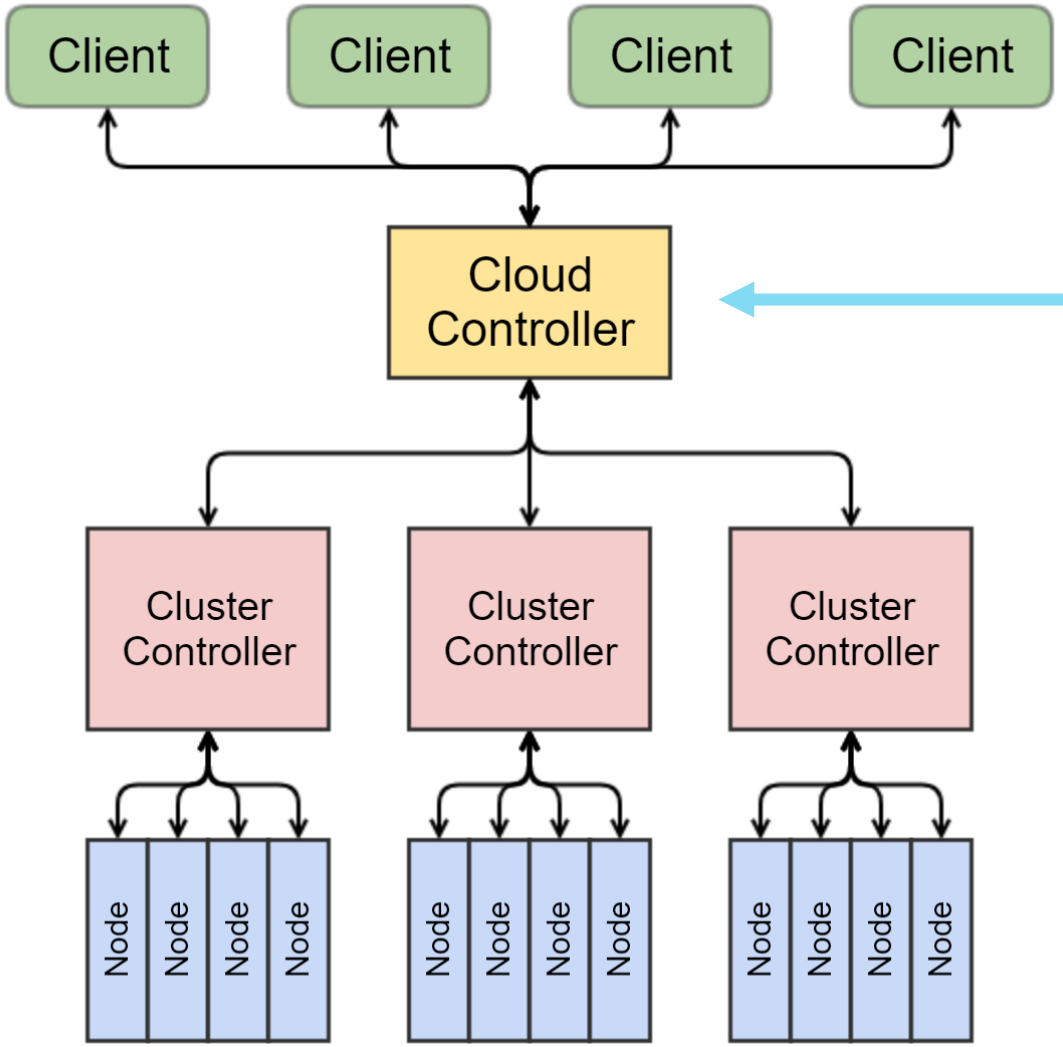
FaaS – Real World Examples

- Amazon Lambda
  - Supports Java, Node.js, C#, and Python
- Google Cloud Functions
  - Supports Node.js
- Azure Functions
  - Support .NET, Java

- Upload code and link it to triggers
- Triggers could be other Web Services in the same Cloud or triggered through API endpoints
- No control over the platform or infrastructure
- Function is deployed and run when/where required
- Function executes to end then dies (non-persistent, may take some time to start)
- Generally pay depending on resources used over time
  - E.g. GB of memory per second of use

UNIVERSITY of HULL

Example (basic) Cloud Architecture



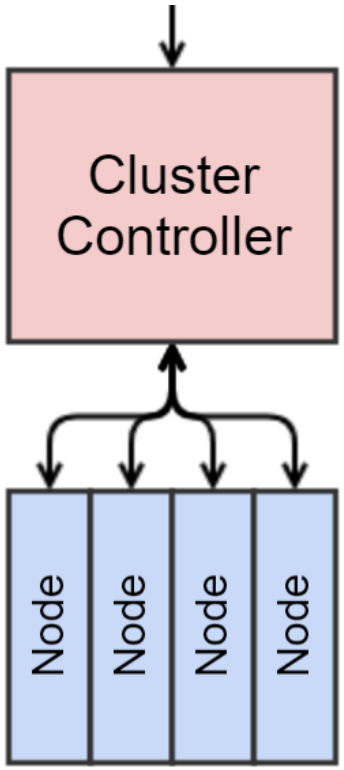
Likely to be lots of steps rather than one machine (e.g. routing, filtering, load balancing)

May be required to reroute to another cloud controller (e.g. in another part of the world) if necessary

UNIVERSITY of HULL

Physical Clusters

- A cluster is typically a number of servers working together that appear (to the system) to be a single system
- Typically connected together with very fast LAN connections, these clusters can operate together to provide a very high level of concurrency
- Cluster Controllers allow the cluster to appear as a single unit, manage the dispersion of tasks and facilitate routing to individual nodes.
- Possible to have clusters of clusters to ease the load on each controller or increase the granularity of the system
- Each node could run many, many virtual servers/functions
  Or could be allocated to a single, very resource-hungry purpose
- If more than one node used at the same server farm for the same purpose it often makes sense to host it on co-located nodes (easier routing)



UNIVERSITY of HULL

PaaS/FaaS Computing Problem

- Problem: if you write code it comes with lots of dependencies
  - Operating system (and version)
  - Runtime (and version)
  - Libraries (and versions)
  - Filesystem/Folder structure
  - Security policies
  - Etc.

- IaaS uses a virtual machine or similar where you get lots of control – so you can ensure the environment is correct – *but getting it set up or to automatically scale can be a pain*
- PaaS and FaaS don't provide you with a virtual machine for your use, so how do they load and manage all of the parts of the applications that need to run on a single machine?
- How can you easily scale up to more servers quickly without setup time overhead?
- Also, how are functions or applications sent to a server for processing if it is not known which server they will run on at deployment?

UNIVERSITY of HULL

Solving the PaaS/FaaS Computing Problem

One option to get around this is to only support a small number of configurations

E.g. Only support applications which run

- On Windows Server
- Using a few selected frameworks
- And few programming languages
- With standard/default settings and configuration

However, if you want to permit more flexibility…

The answer is containers

UNIVERSITY of HULL

Containerised Computing

- A container packages up the dependencies for a single application or function into a single executable block
- When executed, this block replicates the exact environmental conditions required for the code to run – no matter what machine it is running on

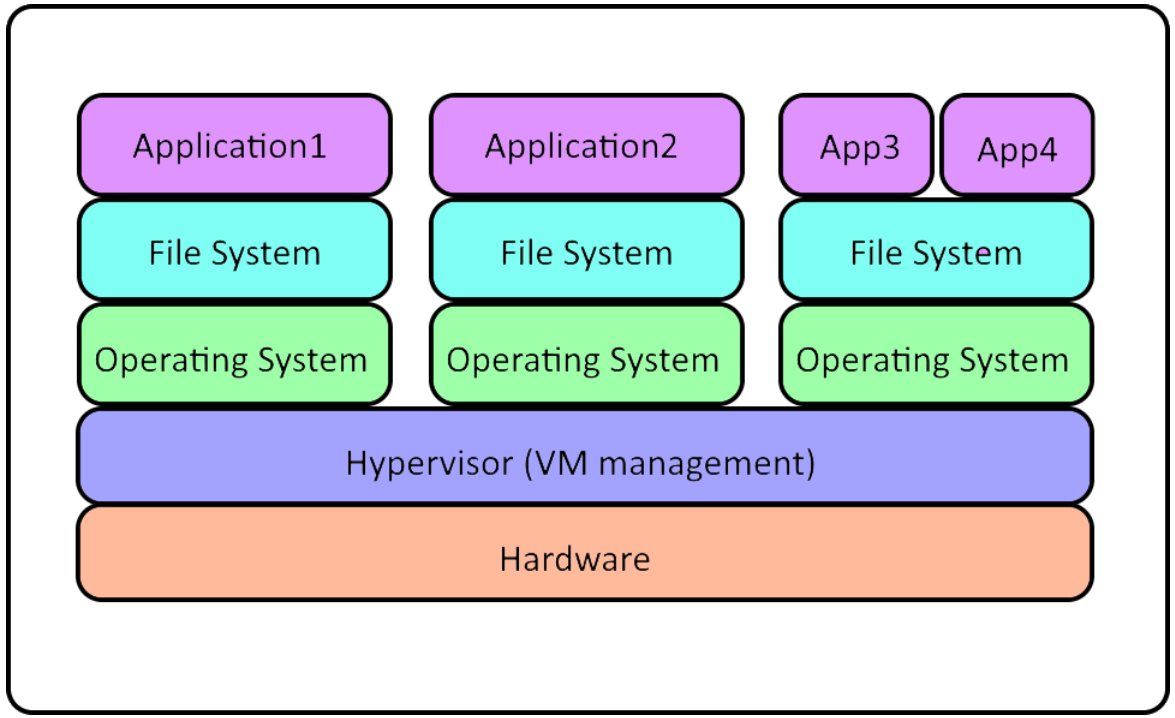
- Blocks in containers can be:
  - Run on different underlying hardware
  - Run on different underlying operating systems
  - Started much faster than a VM (and are self-configuring)

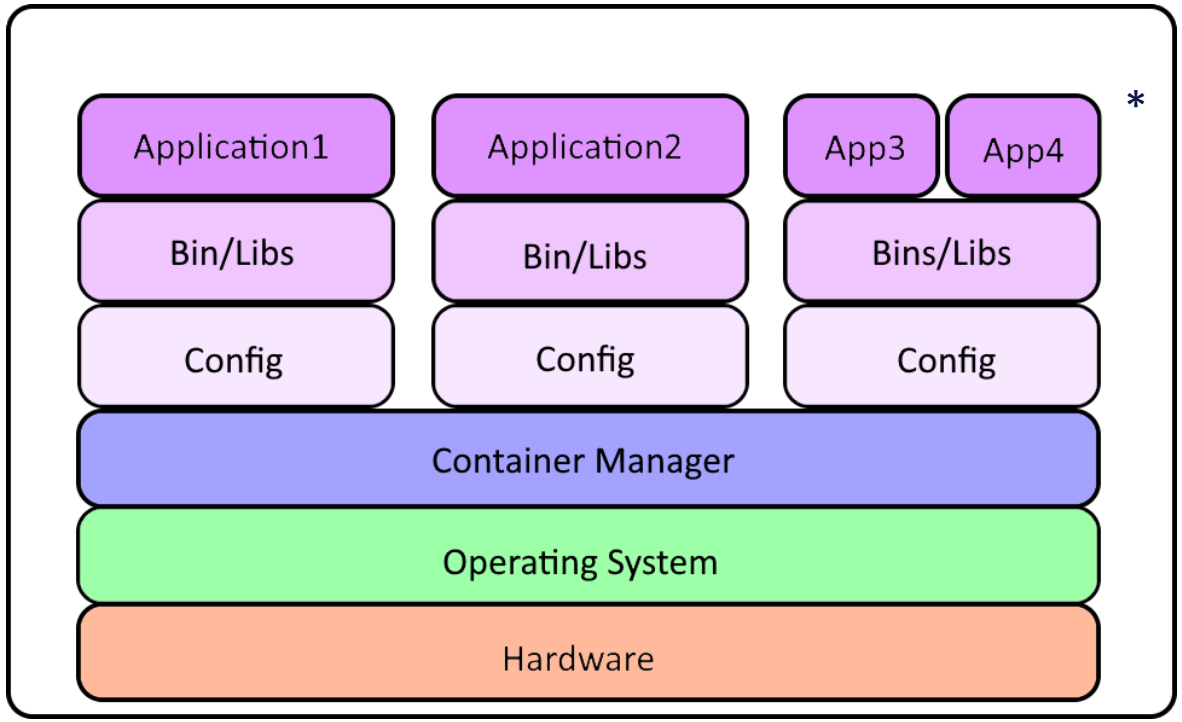
- However, they must run within a container management environment
  - E.g. Docker, Kubernetes

- Containers can be created and controlled by the developer or by the host

UNIVERSITY of HULL

Containerised Computing Vs. Virtual Machines





*Multiple apps/functions is common for a VM but uncommon in a container (although possible)

UNIVERSITY of HULL

.Net Core

A significant goal for the .Net Core team was to move towards self-contained applications that were highly portable without the need to run on specific devices or operating systems.

That is to say that all of the necessary dependencies can be easily deployed as part of the publishing process. This includes:

- required libraries
- the runtime
- configuration details
- any framework or host (e.g. Kestrel as a lightweight, platform-independent host)

This goal has generally been met and thus .Net Core applications can more easily be deployed to servers and containers without having to install additional components (e.g. the .Net Framework Runtime) or configure the environment manually.

High-Power Computing

- Clearly The Cloud has a **huge** amount of processing power and storage

- Strangely Clouds can sell more processing power than they actually have available
  - Not all code is likely to run simultaneously
  - Not all code will fully utilise the max resources available to it
  - With a good management layer, Clouds can manage the resources dynamically to repurpose processing power (rather than letting components sit idle).

- High-power applications can make use of The Cloud as a pre-built processing supercomputer
  - Resources are flexible to meet computing needs
  - No maintenance, replacement or upfront cost
- May, however, be slower than a supercomputer
  - General Purpose CPUs vs Specialised Processors
  - Application is spread across multiple machines rather than multiple cores (messaging is slower)

UNIVERSITY of HULL

High-Power Computing

- Examples:
  - Artificial intelligence (AI) and algorithms that learn (e.g. genetic or neural)
  - Big data analysis, image processing and data amalgamation
  - Speed focussed applications

Case Study:

- Self-driving cars have onboard computers to analyse the road/car conditions and make decisions at very fast speeds.
  - Onboard computers are limited by: power, size, storage capacity, number of possible concurrent processes
  - Also these computers quickly become outdated
- One option is to connect the car to The Cloud and use the practically unlimited power, concurrent processes and storage available. The Cloud can also be updated easily to improve all cars' performance and ensure the newest technology is being utilised without cars having to be brought in to make changes.
- The system could also allow cars to cooperate, making he roads safer

- ***What are the problems??***

UNIVERSITY of HULL

Cloud Car Case Study

- Problems:
  - Cars may lose connectivity → Solution: improve mobile network availability/stability
  - Security issues → Solution: heightened security, DDoS prevention, safe failure modes
  - Non-specific hardware → Solution: Cloud set up for this specific purpose

  - Data quantity problems
    - Cars collect a huge amount of data for processing (quantity likely to grow as cars become 'cleverer' and components cheaper)
    - Lots of data to send to a data centre as the car is driving
    - Possible overloading issues where lots of cars are involved

  - Speed issues
    - Messages need to get to/from a remote data centre
    - Then messages need to be routed within the data centre to the specific server that is managing the request
    - Services/applications/functions may need to be deployed/"spun up", taking time
    - Car needs a FAST response

Edge Computing

Cloud computing data centres have been built mostly in out-of-the way locations

- Cheap land
- No neighbours who might dislike the huge buildings/machinery
- Lots of space for expansion/power generation/power storage

Edge Computing is a move to bring some of the computation closer to the source of the data

- Performing calculations closer to the data source
- Reduces the number of messages that need to be sent on a network-scale
- Reduces the calculations that need to be done by a centralised machine
- Increases the speed at which responses can be generated, sent and received
- Edge computation can be done on hardware/software designed for the purpose

- The network infrastructure connecting the 'Edge' to the 'Cloud' is called the **'Fog'**

UNIVERSITY of HULL

The Fog

If the Cloud is a long way away, slow and large…

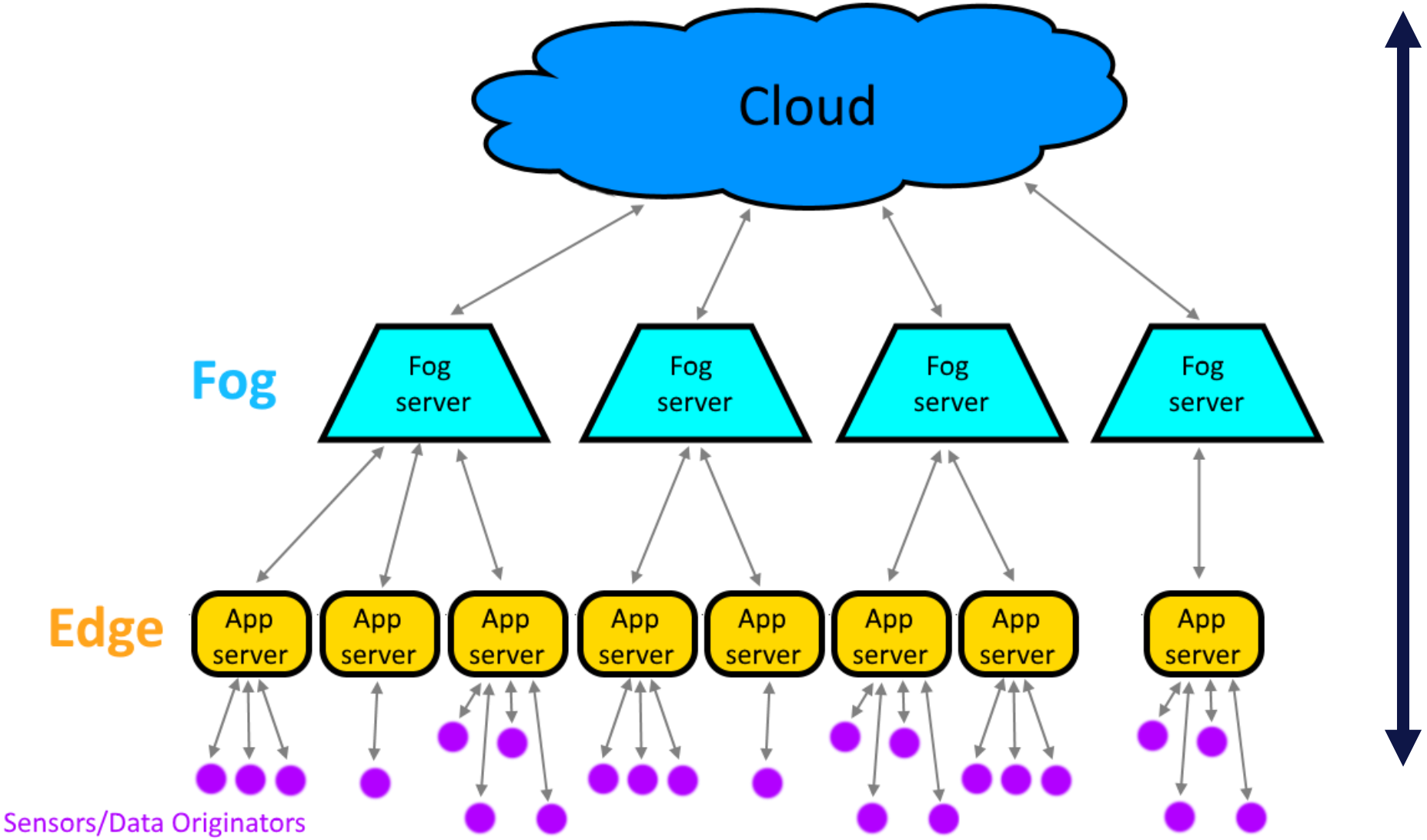
…and the Edge is close, fast and small

The Fog offers an in-between, providing higher-power, medium distance compute services to many nearby Edge servers.

The Fog is what can make Edge Computing so powerful and so often distributed, hierarchical computing is just called ‘The Fog’ to differentiate it from simply ‘The Cloud’

UNIVERSITY of HULL

Edge Computing



Slow Response
High Performance

Fast Response
Low Performance

UNIVERSITY of HULL

Using Edge Computing for our Case Study

Cars make basic calculations and judgements using onboard hardware

- Very, very fast response – local computing power
- Low performance (less able to make complex calculations at speed)
- Good enough for non-critical decisions
- Useful to have a critical decision made in case response is too slow
- Decisions/calculations are local to car

Cars filter and send relevant data to Edge nodes

- Very fast response
- Edge nodes are located physically close to cars (roadside or near to mobile data routers)
- Specialised hardware/software is designed to make decisions at speed using lots of data
- Finite computing power
- Decisions and calculations are local to nearby cars

Edge nodes filter and send relevant data to Fog

- Fog servers amalgamate data

Fog servers filter and send relevant data to Cloud

- Slow response
- Nearly unlimited data storage and power
- Decisions/calculations are global

UNIVERSITY of HULL

Some Cloud Service Benefits

- IaaS
  - Reduced capital investment with only minimal loss of control (of hardware)
  - Scalability
  - Resilience
  - Metered costs
- PaaS
  - Reduced management overhead, no need to think about environment (OS, for example)
  - Backup safety?
- FaaS
  - No management overhead
  - Available on demand
  - Priced based on actual processor time
- SaaS
  - Many applications, no need for local install
  - Possible to use cheaper client devices
  - No upgrade cycles
  - Reduced management
  - Reduced localisation of workforce

UNIVERSITY of HULL

Some Cloud Benefits

- Cost
  - Licence fees, pay as you go
- Storage
  - Effectively unlimited
- Power
  - Huge availability for distributed concurrent computing
- Scalability
  - Can be automatic
- Backup
  - Not guaranteed, but easy to set up
- Data accessibility
  - Internet is everywhere
- Speed of deployment
  - No hardware or infrastructure procurement
- Maintenance
  - Handled by someone else

UNIVERSITY of HULL

Some Potential Cloud Drawbacks

- Reliability
  - You haven't got control over the hardware
  - It's likely that large Cloud providers have lots of clients – are you at the top of their support list?
- Security
  - Who can see your data?
  - Who has access to your code?
  - Single point of attack may affect 10,000s of products
- Visibility to attack
  - No corporate firewall
  - Unknown firewalls and security policies
- Unpredictable costs?
  - Based on usage/demand, which may be out of your control
- Speed
  - Lots of additional routing
  - "Spin-up" time required, especially for smaller dynamic services (e.g. FaaS)
  - (Possibility of using edge computing to offset this)

UNIVERSITY of HULL

Example Major Cloud Outages

- Google Docs
  - Outage for an hour affecting many users
  - No access to files for business and home users
  - Also affected Google's own teams
- Microsoft Azure
  - Outage for over 7 hours
  - Air conditioning malfunction caused machines to overheat
  - Affected the majority of Cloud services in Europe
- Amazon Web Services
  - Data storage errors
  - Lasted for about an hour
- Apple iCloud
  - No backups for ~1% of users (still a huge number)
  - Lasted for 36 hours
- Microsoft Office 365
  - Authentication errors for email users
  - 1 day downtime

- Google Cloud
  - Database access errors
  - Affected apps reliant on low latency, e.g. Games
  - Lasted for 12h
- Microsoft Azure
  - Storage errors for a few hours
  - Caused by a cluster losing power
- Amazon Web Services
  - 'The Big One'
  - An incorrect command typed by an engineer took the entirety of AWS down for just over 4 hours
  - Affected every website/API/app/service using AWS
  - This is about 1/3 of the global Cloud market
- IBM Cloud
  - Control & Management interface inaccessible
  - Lasted for several hours
  - Caused by an update gone wron

UNIVERSITY of HULL

Expanding on Topologies (Homework!)

UNIVERSITY of HULL

Expanding on Topologies

Important considerations for different architectural topology options

Independent Learning

UNIVERSITY of HULL

Centralised Architectures

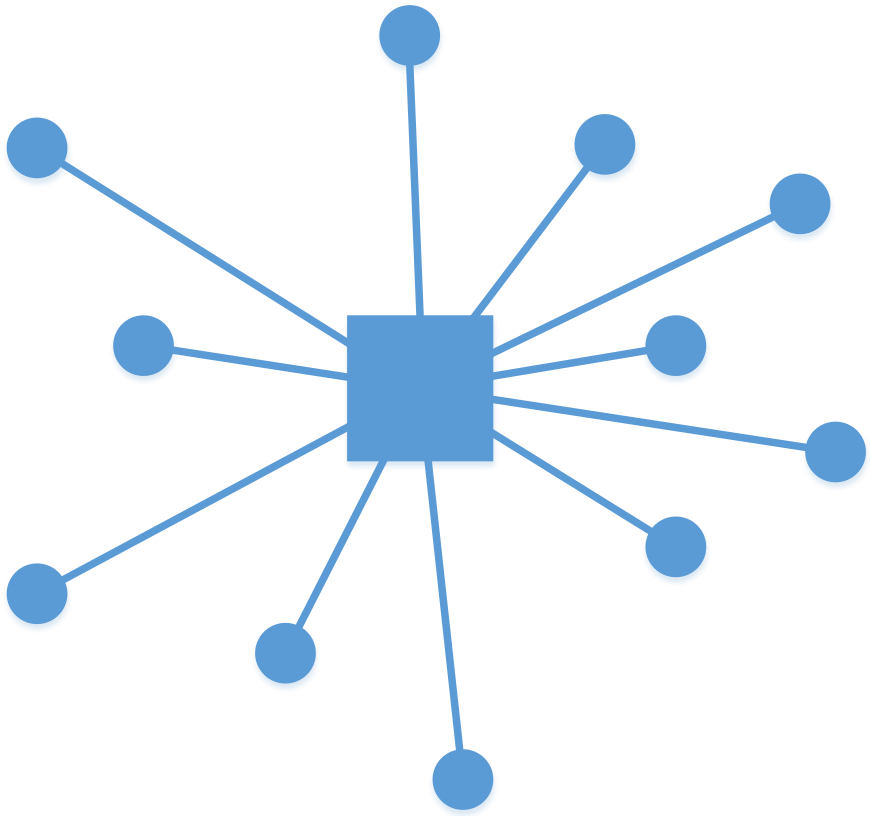
A centralised architecture uses a single central hub with many 1-to-1 connections

- or transparency that acts like 1-to-1 connections

- The hub usually manages incoming/outgoing transfer with the nodes/leaves/clients

- This means that the hub has ultimate control over what happens
  - Simple architecture – easy to keep on top of what is connected
  - Easy to perform updates and track errors etc.
  - Hub does all routing/management activities – nodes can be basic

- But…
  - Hub must be very powerful
  - Hub may become congested
  - Hub must be able to talk to all nodes and perhaps perform translation
  - Nodes have no independence – single point of failure



UNIVERSITY of HULL

Centralised Physical Systems

- Physical systems using a Centralised Architecture are common

- Example:
- Web Server:
  - If we concentrate on the server/client relationship, a website server is usually a Centralised System
  - That is, a central server has many 1-to-1 connections with clients
  - Clients are thin, browser-based front-end devices only
  - The server accepts requests and makes responses
  - If two clients want to communicate, they can – but must do so through the central web server

- This is not to say that the Internet is a Centralised System – it is not!
  - But when we concentrate on the web server/client relationship, it exhibits a centralised architecture

UNIVERSITY of HULL

Centralised Applications

Centralised applications may be distributed over many devices over the Internet

- The devices need not be assembled using a centralised architecture (although it may be useful)

OR a centralised application could be running on one machine

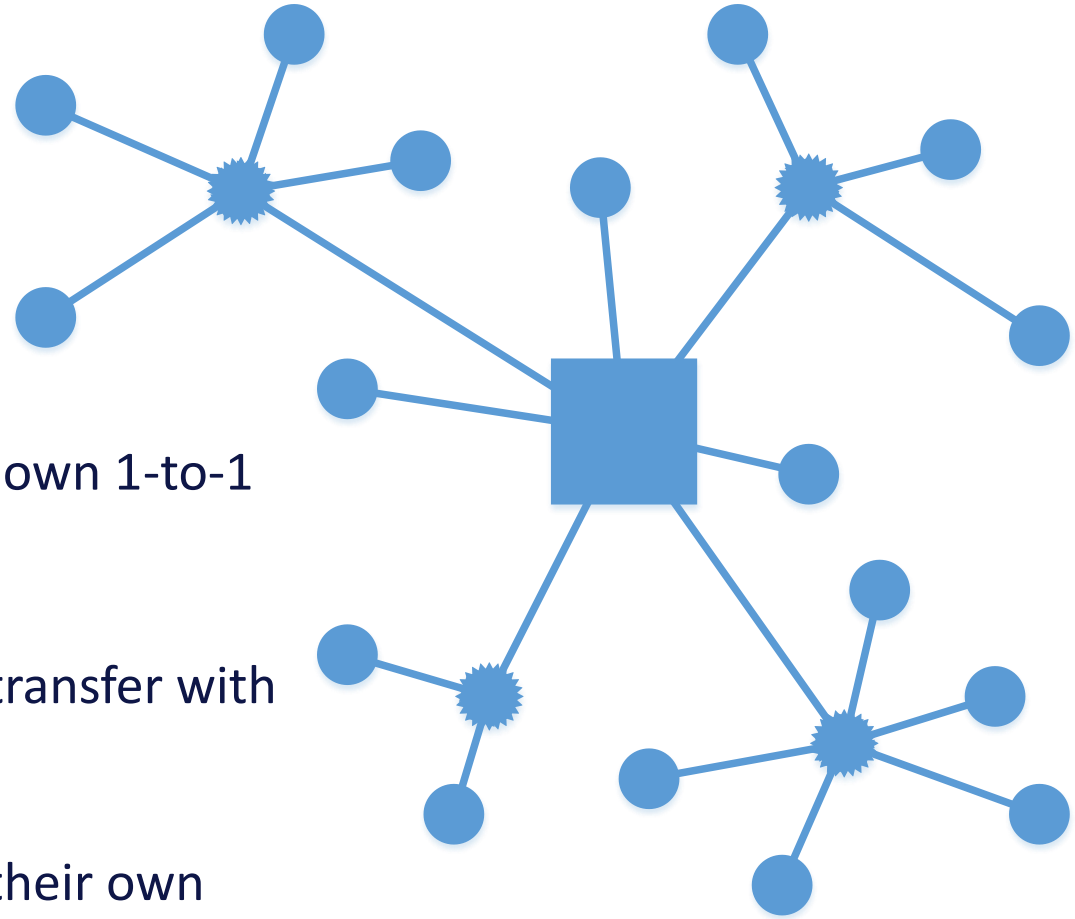
A single controlling part of the application manages all other parts

- Could be a control/parent thread that uses other threads to do the work
- OR could be services/application parts running on remote machines that all communicate with a single point

Strangely – whilst a web server can exhibit a centralised physical architecture, a web client can exhibit a centralised application architecture

- If it is using services/application parts from many servers and drawing these outputs together into one application

Decentralised Architectures



A decentralised architecture uses a single central hub with many 1-to-1 connections

BUT connected nodes may also be a secondary hub for their own 1-to-1 connections

- The central hub *may* manage all of the incoming/outgoing transfer with all nodes/leaves/clients
  - Secondary hubs route everything back to the main hub
- OR the secondary hubs may be able to do some routing of their own
  - Between nodes they have a direct connection with

- The hub may not need to do all the processing or manage all data
- BUT it loses some control and secondary hubs must be that much more powerful to manage many connections/routing

UNIVERSITY of HULL

Decentralised Physical Systems

Example:

- Intranet with routers and switches

Likely to have a common central point (usually where the network attaches to the Internet)

Smaller routers, access points and switches attach to this common central point

Devices connect to routers/access points/switches

- Central point provides connectivity to Internet for all
- Routers/access points/switches manage routing of intranet data and manage new connections
  - Data may or may not need to go through central router for intranet
  - Data will need to go through central router for Internet

UNIVERSITY of HULL

Decentralised Applications

Again, decentralised applications can be designed to work with or without a decentralised physical topology.

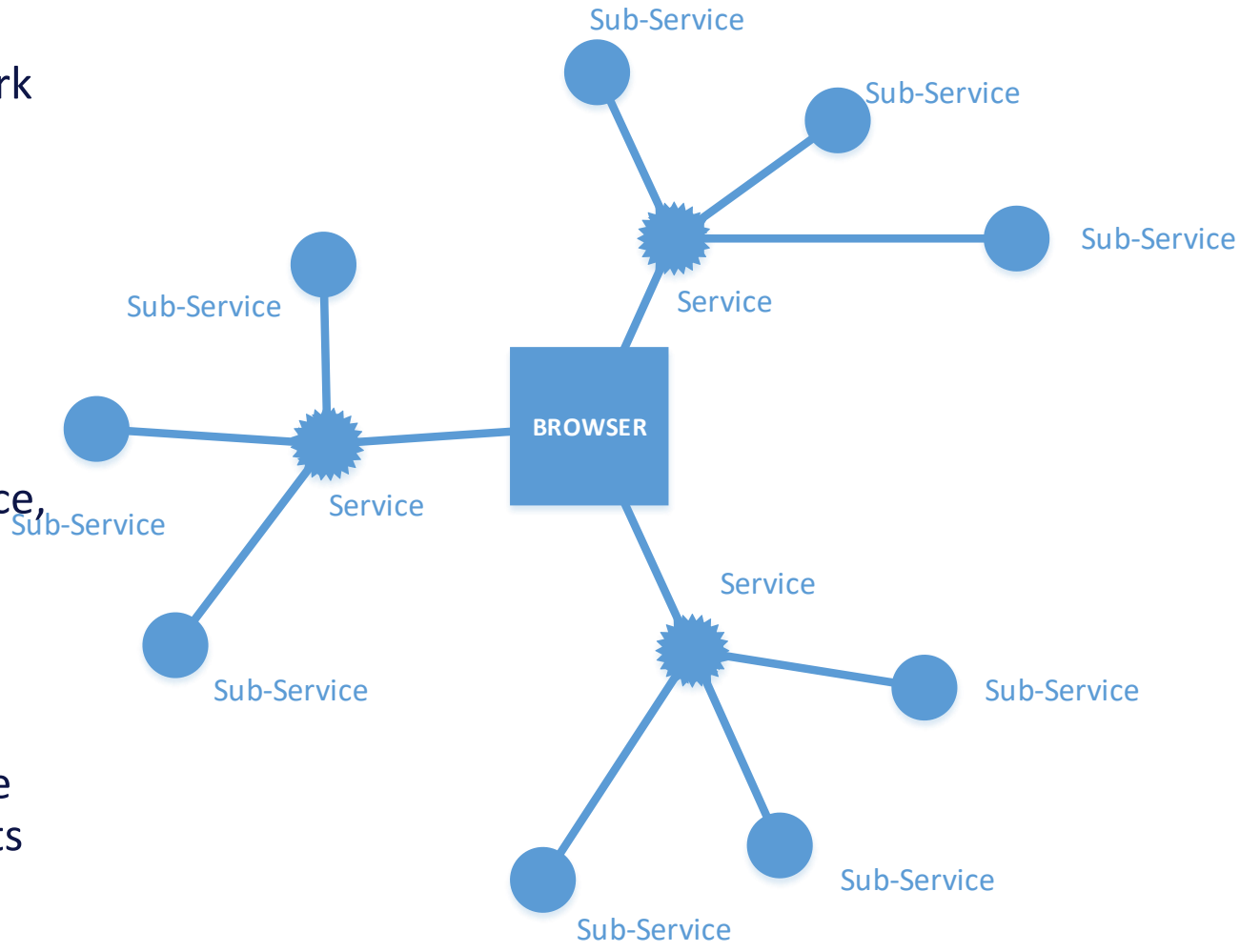
Decentralised applications may be designed to operate over the network or not.

If an application uses a service which uses another service, this can be considered a decentralised application.

Example:

Browser requests resources from 3 services, each service requests resources from three further sub-services… (lots of delegation)

3-levels of hierarchy in this example



UNIVERSITY of HULL

Distributed Topology

No common central point

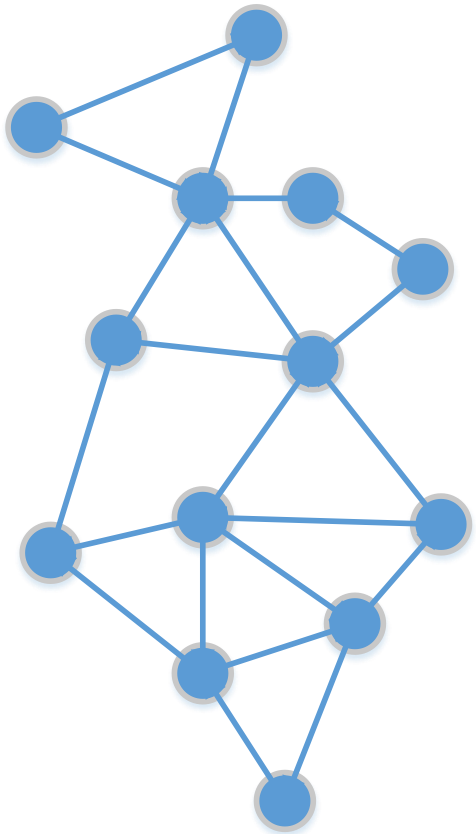
All devices manage routing and no one ‘hub’ has control.

An important topology but difficult to manage

- All devices must be able to route/manage connections
- Difficult to perform updates, track errors

- Good where it would be impossible/difficult to use a central hub
- Good for connecting other systems

- Example: Driverless smart cars that can interact to prevent accidents
- Example: Application maximising optimal resource sharing



UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Identifying what The Cloud really is and a basic Cloud architecture
- Some real-world Cloud service providers
- Containerised deployment
- Edge computing and The Fog
- Benefits and drawbacks of The Cloud

In the next (and final) session we will finish the module by reviewing the content as exam prep.

We'll also have a look at an example exam question.

UNIVERSITY of HULL